INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Bahia

touying-ifba-unofficial

Lecture slides for the Federal Institute of Bahia (IFBA).

v0.1.0 October 03, 2026

https://github.com/lucaslrodri/touying-ifba-unofficial

Lucas Lima Rodrigues

A Touying theme for lecture slides, following the visual identity of the Federal Institute of Bahia (IFBA). Slides are written with headings and a few Markdown-like shorthands for boxes, quotations, arrow lists and emphases. This manual is set with the rules of the theme itself.

Contents

1 Quick start 2
1.1 The information of the deck 2
1.2 The theme and the cover 3
1.3 Language 4
1.4 Dependencies 5
2 Slides and headings 6
2.1 Sections 6
2.2 Slides and continuations 6
2.3 Exercises and examples 7
2.4 Subtitles 9
2.5 Slide functions 10
3 Inline formatting and shorthands 14
3.1 Bold, italic and alert 14
3.2 Highlight and underline 14
3.3 Code chips 14
3.4 Colors by context 14
3.5 Colored emphases 15
3.6 Escapes 16
4 Lists and terms 17
4.1 Arrow lists 17
4.2 Bullet lists 17
4.3 Numbered lists 17
4.4 Terms 18
5 Boxes and quotes 19
5.1 Emphasis boxes 19
5.2 The box shorthand 19
5.3 Equation boxes 20
5.4 Quotations 21
6 Columns 22
7 Colors and logo 23
7.1 The logo 23
8 Tables, figures, math, code and icons . . 26
8.1 Tables 26
8.2 Figures and math 26
8.3 Code 26
8.4 Icons 27
9 Touying features 30
10 Index of definitions 32

1 Quick start

The theme is not on Typst Universe: it is installed into the `@local` namespace of Typst by a script of the repository, which also offers to install the font.

```
# macOS and Linux
curl -fsSL https://raw.githubusercontent.com/lucaslrodri/touying-ifba-unofficial/main/scripts/install.sh | sh
# Windows (PowerShell)
irm https://raw.githubusercontent.com/lucaslrodri/touying-ifba-unofficial/main/scripts/install.ps1 | iex
```

A deck then imports Touying and the theme, applies `ifba-theme` and usually starts with the cover:

```
#import "@preview/touying:0.7.4": *
#import "@local/touying-ifba-unofficial:0.1.0": *
```

```
#show: ifba-theme.with(
  aspect-ratio: "16-9",
  lang: "en",
  config-info(
    subject: [Neural Networks],
    subtitle: [Units III and IV],
    title: [Training and evaluation],
    author: [Prof. Author Name],
    subject-code: [ENG101],
    counter-prefix: [1.],
  ),
)

#title-slide()
```



`typst init @local/touying-ifba-unofficial:0.1.0` creates a deck from the package template, and `example/main.typ` shows the elements of the syntax in a short deck. From here on, every example shows the source on the left and the result on the right: a real slide when the feature belongs to the slide (headers, pills, continuations), otherwise the content alone, laid out by the same rules as in a deck.

1.1 The information of the deck

`config-info` is Touying's; the theme reads these fields:

Field	Where it appears
`subject`	main line of the cover (large, white) and left side of the footer
`subtitle`	above the subject on the cover, in light amber small capitals
`title`	below the subject on the cover, in white
`author`	bottom of the green panel of the cover and right side of the footer
`subject-code`	before the subject in the footer: "CODE · SUBJECT"
`counter-prefix`	prefix of the exercise and example numbers: `[1.]` gives "Exercise 1.N", `none` gives "Exercise N"

1.2 The theme and the cover

`ifba-theme()`

```
ifba-theme(
  aspect-ratio: "16-9"  str
  lang: "pt-br"  str
  exercise-name: auto  auto  content
  example-name: auto  auto  content
  part-name: auto  auto  content
  footer-left: auto  auto  content  function
  footer-right: auto  auto  content  function
  ..args  dictionary
  body  content
) -> content
```

The theme. It sets the page (20pt Open Sans, the IFBA header and footer) and every rule of the syntax, and takes Touying's configs as extra arguments.

```
#show: ifba-theme.with(
  aspect-ratio: "16-9",
  config-info(
    title: [Training and evaluation], subtitle: [Units III and IV],
    author: [Prof. Name], subject: [Neural Networks], subject-code: [ENG101],
    counter-prefix: [1.],
  ),
)
```

`config-info` carries `subject`, `subtitle`, `title`, `author`, `subject-code` and `counter-prefix`, the prefix of the exercise and example numbers ("Exercise 1.N"; `none` gives "Exercise N").

Argument

`aspect-ratio: "16-9"` — `str`

Any ratio accepted by Touying's `utils.page-args-from-aspect-ratio`: `"16-9"`, `"4-3"`, ...

Argument

`lang: "pt-br"` — `str`

Language of the deck: `"pt-br"` (or `"pt"`), `"en"` or `"es"`, optionally with a region (`"en-US"`). It sets `text.lang` / `region` and the localized names below.

Argument

`exercise-name: auto` — `auto` | `content`

Name in the pill of exercise slides; `auto` is Exercício / Exercise / Ejercicio.

Argument

`example-name: auto` — `auto` | `content`

Name in the pill of example slides; `auto` is Exemplo / Example / Ejemplo.

Argument

`part-name: auto` — `auto` | `content`

Prefix of the section slide ("Part N:"); `auto` is Parte / Part / Parte.

Argument

`footer-left: auto` — `auto` | `content` | `function`

Left text of the footer; a function receives `self`. `auto` is the subject code, a dot and the subject (the dot only when both are given).

Argument

`footer-right: auto` — `auto` | `content` | `function`

Right text of the footer; a function receives `self`. `auto` is the author.

Argument

`..args` — `dictionary`

Touying configs, typically `config-info(..)`; also `config-common(..)`, `config-page(..)`.

Argument

`body` — `content`

The deck.

`title-slide()`

```
title-slide(
) -> content
```

Cover: a green panel with a rounded corner on the left, with the subtitle (light amber small capitals), the subject (large, white) and the title (white) of `config-info`, and `footer-right`/`footer-left` (the author and "CODE · SUBJECT") at the bottom; on the right, on white, the complete IFBA logo over a line art of rounded squares. The slide counter is frozen, so the cover does not count.

1.3 Language

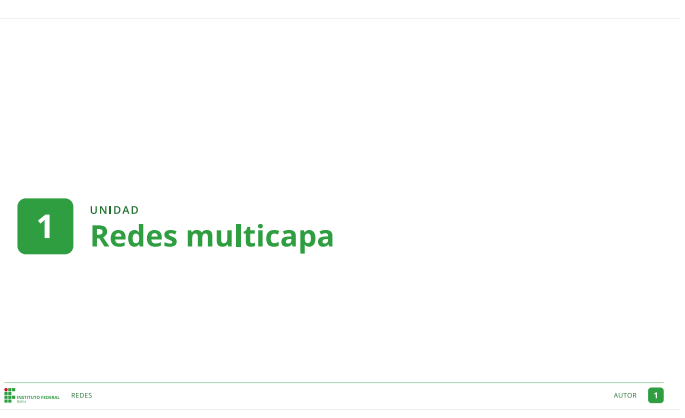
The theme speaks Portuguese by default (`lang: "pt-br"`): Exercício, Exemplo, Parte. `lang: "en"` or `"es"` switches the text language and those three names, and each name can still be overridden.

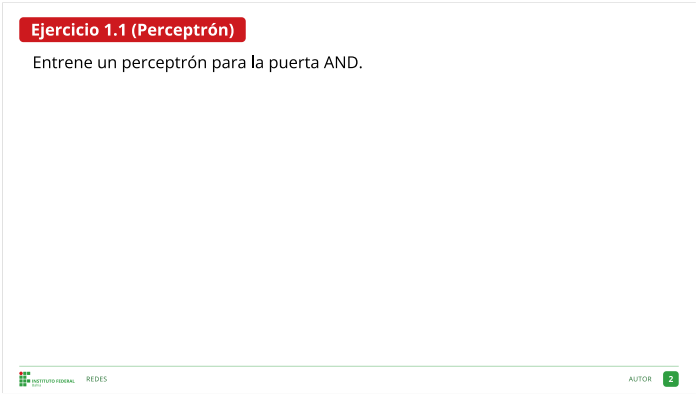
```
#show: ifba-theme.with(
  lang: "es",
  part-name: [Unidad],
  config-info(subject: [Redes],
  author: [Autor]),
)

= Redes multicapa

== Perceptrón <exercise>

Entrene un perceptrón para la
puerta AND.
```





1.4 Dependencies

The versions below are read from the sources of the package, so this table cannot go stale. All of them are the latest releases at the time of writing and are fetched automatically by Typst.

Package	Version	Role
`codly`	`1.3.0`	code blocks; its functions are re-exported by the theme
`linguify`	`0.5.0`	localized names (Exercise, Example, Part)
`touying`	`0.7.4`	the slide engine; the deck imports it too, with the same version

Besides the packages, the theme needs **Typst 0.14.0 or later** and the font **Open Sans**, the typeface of the brand of the Institutos Federais, installed on the system (it is not bundled with Typst, unlike **New Computer Modern Math**, used for the equations; the installer downloads it with `--fonts`). Building this manual also takes tidy 0.4.3, and the test suite runs on Tytanic 0.4.1.

2 Slides and headings

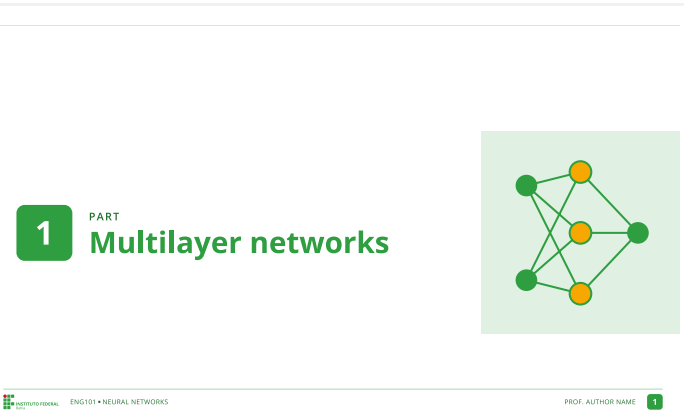
Headings drive the deck: no function call is needed for the ordinary slides.

Markup	Effect
`= Title`	section slide: the number of the part in a green square, “Part” and the title; what follows the heading goes to the right
`== Title`	new slide with a green header, fitted to the width
`== Title <exercise>`	exercise slide: red pill “Exercise 1.N (Title)”, numbered automatically
`== Title <example>`	example slide: blue pill “Example 1.N (Title)”, with its own counter
`=== Subtitle`	green pill; no new slide; repeated at the top of every continuation page
`==== Subtitle`	the same pill, not repeated
`---`	alone on a line: next page of the same slide (same header, same number)

2.1 Sections

The content right after a `=` heading, typically an image, sits on the right, flush with the page edge. `#pad(right: 1.8em)[#image(..)]` restores the page margin.

```
= Multilayer networks
#image("figure.svg")
```



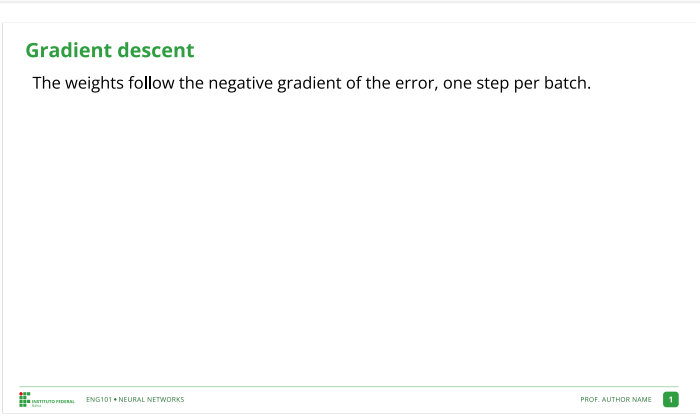
2.2 Slides and continuations

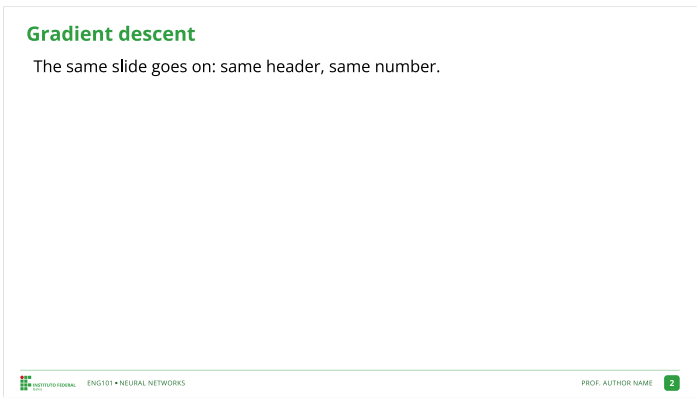
```
== Gradient descent

The weights follow the negative
gradient of the error,
one step per batch.

---

The same slide goes on: same
header, same number.
```





Caution

Touying breaks the page at *any* top-level em dash, so an em dash next to inline markup, as in `Text – *bold*`, also starts a new page. Use an en dash or wrap the paragraph in `#[ ]`.

2.3 Exercises and examples

A label on the `==` heading turns the header into a numbered pill. Any suffix gives the same slide and makes the label unique, so the slide can be cited: `@exercise-ogata` becomes “Exercise 1.N”, bold, in the color of the kind and linked to the slide. `#exercise-slide` and `#example-slide` are the function forms, with a `color` option; they cannot be cited.

```
== Ogata 4.6 <exercise-ogata>

Find the transfer function of the system.

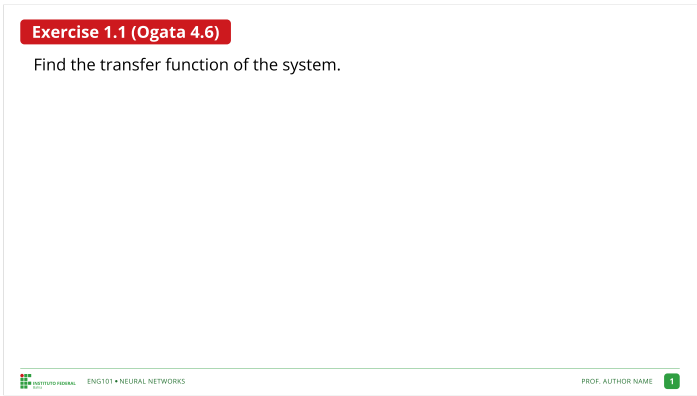
== XOR network <example>

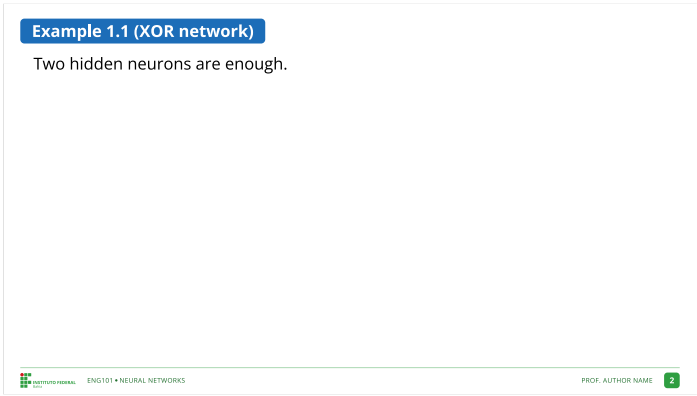
Two hidden neurons are enough.

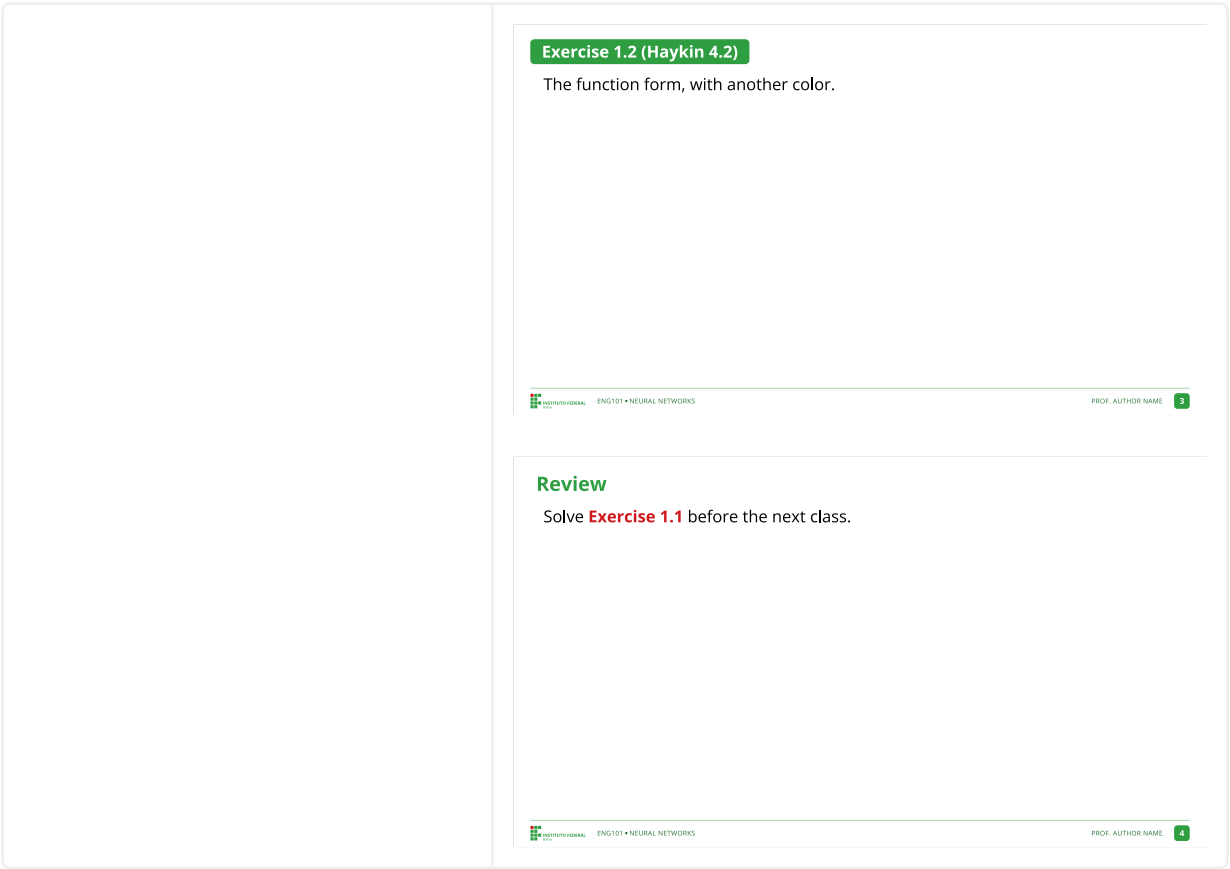
#exercise-slide(title: [Haykin 4.2], color: colors.primary)[
  The function form, with another color.
]

== Review

Solve @exercise-ogata before the next class.
```







`exercise-slide()`

```
exercise-slide(
  title: [] content
  color: auto auto color
  body content
) -> content
```

Exercise slide: the header becomes a pill “Exercise 1.N (Title)”, numbered automatically. The function form of `== Title <exercise>`. Only the heading form can carry a unique label (`== Title <exercise-ogata>`) and be cited with `@exercise-ogata`, which shows “Exercise 1.N” linked to the slide.

```
1  #exercise-slide(title: [Ogata 4.6])[Find the transfer function.]
```

Argument

`title: []` `content`

Text in parentheses after the number; empty omits the parentheses.

Argument

`color: auto` `auto` | `color`

Color of the pill; `auto` is red (`tertiary`).

Argument

`body` `content`

Content of the slide; `---` inside it continues the same exercise.

`example-slide()`

```
example-slide(
  title: [] content
  color: auto auto color
  body content
) -> content
```

Example slide: a pill “Example 1.N (Title)”, with a counter independent from the exercises. The function form of `== Title <example>`; see `exercise-slide()`.

```
#example-slide(title: [XOR network])[Two hidden neurons are enough.]
```

Argument

`title: []` — content

Text in parentheses after the number; empty omits the parentheses.

Argument

`color: auto` — auto | color

Color of the pill; `auto` is blue (`quaternary`).

Argument

`body` — content

Content of the slide.

2.4 Subtitles

A `===` subtitle is repeated on the pages that continue the slide, unless the continuation starts with another `===`. That holds for `---`, for `#pagebreak()` and for content that overflows by itself.

```
== Activation functions

=== Sigmoid

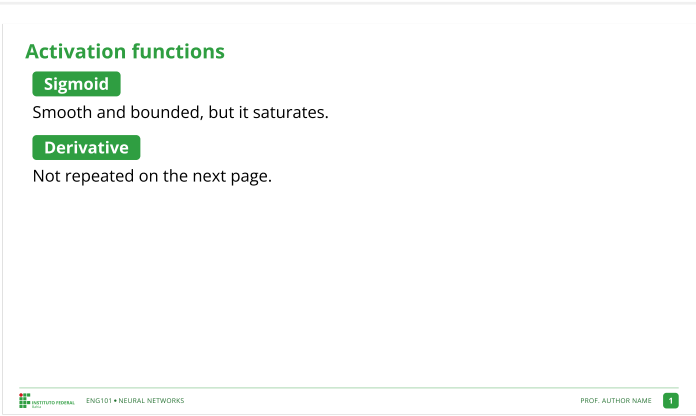
Smooth and bounded, but it saturates.

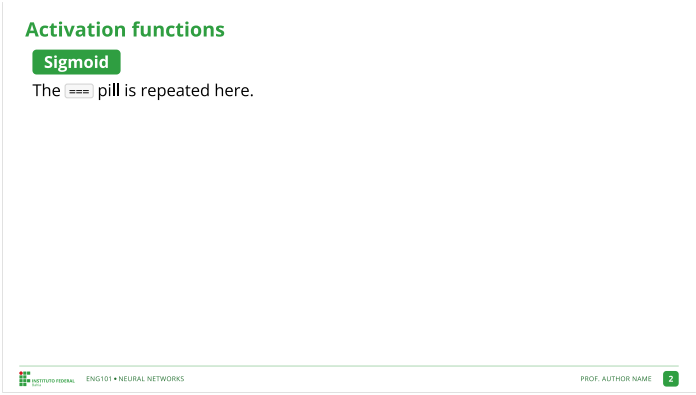
==== Derivative

Not repeated on the next page.

---

The `===` pill is repeated here.
```

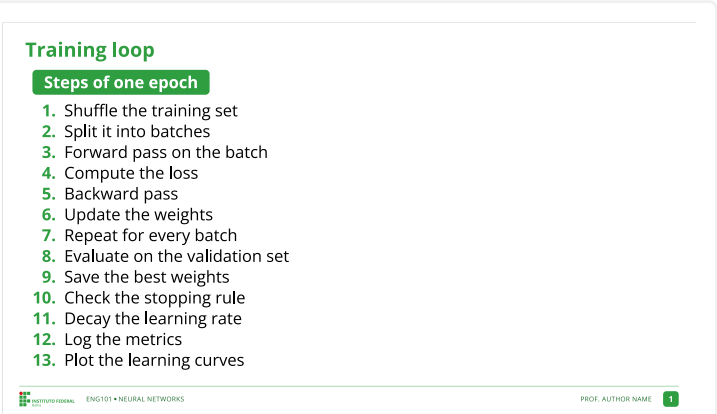


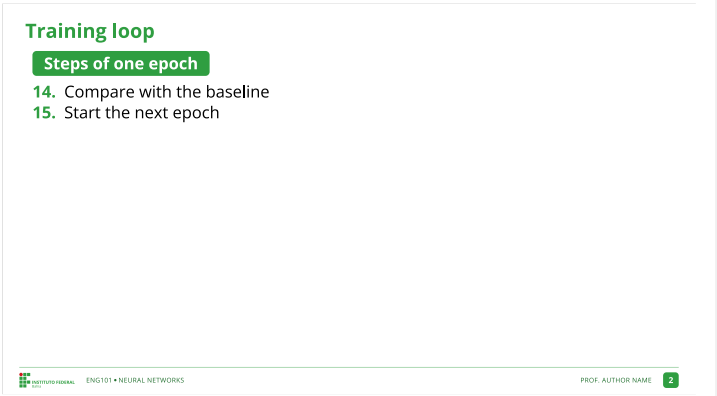


```
== Training loop

=== Steps of one epoch

+ Shuffle the training set
+ Split it into batches
+ Forward pass on the batch
+ Compute the loss
+ Backward pass
+ Update the weights
+ Repeat for every batch
+ Evaluate on the validation set
+ Save the best weights
+ Check the stopping rule
+ Decay the learning rate
+ Log the metrics
+ Plot the learning curves
+ Compare with the baseline
+ Start the next epoch
```





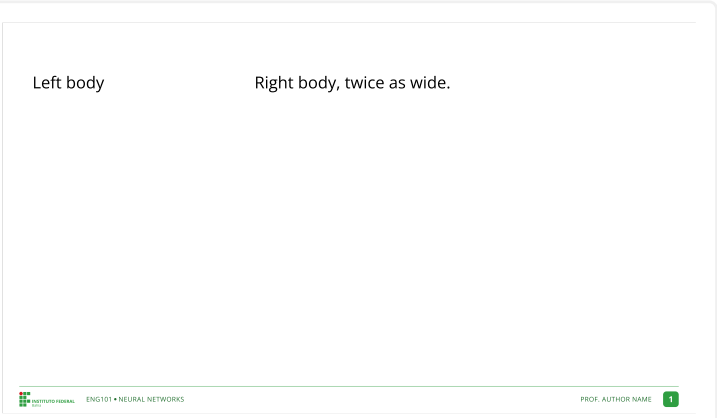
Only a top-level `===` is repeated; inside `#[ ]`, a column or a box the pill stays in the flow. `config-common(breakable: false)` makes Touying warn about overflow instead.

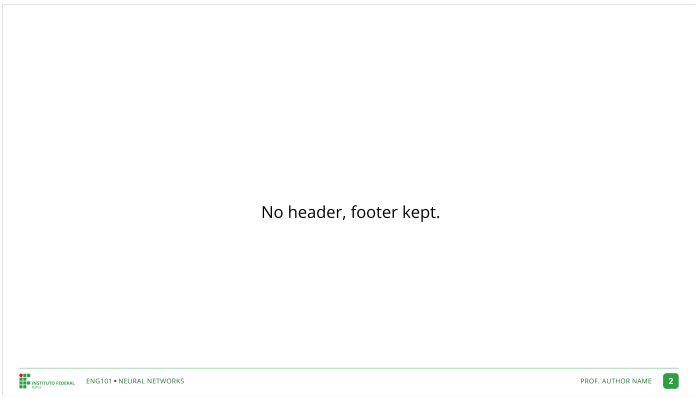
2.5 Slide functions

`#slide` is the slide that every `==` opens; calling it gives access to Touying's arguments, such as `composer`. `#empty-slide` drops the header and keeps the footer. `#title-slide()` is the cover, shown in Section 1.

```
#slide(composer: (1fr, 2fr))[
  Left body
][
  Right body, twice as wide.
]

#empty-slide[
  #align(center + horizon)[No header, footer kept.]
]
```





`slide()`

```
slide(
  config: (:) dictionary
  repeat: auto auto int
  setting: body => body function
  composer: auto auto array function
  ..bodies content
) -> content
```

Default slide of the theme, with header and footer. Every `==` heading opens one, so it is rarely called by hand; the explicit form gives access to Touying's arguments.

A top-level `===` subtitle is repeated at the top of every continuation page of the slide, whether the page comes from `---`, from `#pagebreak()` or from content that overflows by itself, unless the continuation starts with another `===`. A subtitle nested in `#[ ]`, a column or a box is not repeated.

```
1  #slide(composer: (1fr, 2fr))[Left][Right]
```

Argument

`config: (:)` — `dictionary`

Slide configuration: a `config-xxx` dictionary (merge several with `utils.merge-dicts`).

Argument

`repeat: auto` — `auto` | `int`

Number of subslides; `auto` lets Touying count the animations.

Argument

`setting: body => body` — `function`

Set/show rules applied to the slide body.

Argument

`composer: auto` — `auto` | `array` | `function`

Layout of the bodies, e.g. `(1fr, 2fr)`, as in Touying's `side-by-side`.

Argument

`..bodies` — `content`

The slide bodies.

empty-slide()

```
empty-slide(
  config: (:) dictionary
  repeat: auto auto int
  setting: body => body function
  composer: auto auto array function
  ..bodies content
) -> content
```

Slide without the header but with the footer (Touying's `empty-slide` removes both). Same arguments as `slide()`.

Argument

`config: (:)` `dictionary`

Slide configuration, see `slide.config`.

Argument

`repeat: auto` `auto` | `int`

Number of subslides.

Argument

`setting: body => body` `function`

Set/show rules applied to the slide body.

Argument

`composer: auto` `auto` | `array` | `function`

Layout of the bodies.

Argument

`..bodies` `content`

The slide bodies.

new-section-slide()

```
new-section-slide(
  config: (:) dictionary
  level: 1 int
  numbered: true bool
  body content
) -> content
```

Section slide, created by every `= Title`: on the left, the number of the part in a green rounded square, then "Part" in small capitals and the title in green; on the right, the content right after the `=` (typically an `#image`), flush with the page edge. `#pad(right: 1.8em)[#image(..)]` restores the page margin. Not meant to be called by hand.

Argument

`config: (:)` `dictionary`

Slide configuration, see `slide.config`.

Argument

`level: 1` `int`

Heading level, passed by Touying.

Argument

`numbered: true` `bool`

Passed by Touying; the part is always numbered.

Argument

`body` `content`

Content that follows the `=` heading.

3 Inline formatting and shorthands

Most emphases have two forms: a function and a **shorthand**, a short piece of markup that the theme rewrites into the same function. The shorthands take plain text only (no markup inside) and none of them applies inside `raw`.

Function	Shorthand	Result
`#strong[text]`	`*text*`	black bold
`#emph[text]`	`_text_`	black italic
`#alert[text]`, `#primary[text]`	`~~text~~`	green bold
`#highlight[text]`	`==text==`	light-amber background
`#underline[text]`	`~text~`	strong amber underline
`#secondary`, `#tertiary`, `#quaternary`		bold in amber, red, blue
`#emph-box[..]`	`> text`	box, Section 5
`$ #eq-box[$..$] $`	`> $ .. $`	equation box, Section 5
`#quote-box[..]`	`"text"`	quotation, Section 5
`#arrows[..]`	`-> item`	arrow list, Section 4

3.1 Bold, italic and alert

```
Black *bold* and _italic_.

Green #alert[alert], the same as
~~this shorthand~~.
```

Black **bold** and *italic*.

Green **alert**, the same as **this shorthand**.

3.2 Highlight and underline

The `==text==` shorthand cannot open a line (there it is a heading) and its text neither starts nor ends with a space; `#highlight` has no such limits and takes another fill, such as the `-lighter` tones of the palette.

```
A ==highlighted== word and a
#highlight(fill: colors.quaternary-
lighter)[blue one].

A ~strong underline~ and
#underline[the function form]; `Fig.~1`
keeps
its non-breaking space.
```

A highlighted word and a blue one.

A strong underline and the function form `Fig.~1` keeps its non-breaking space.

3.3 Code chips

Inline `raw` is a gray chip. Inside a title or a box it follows the color of the container, and inside a pill it is inverted.

```
Call `fit(x, y)` first.

#emph-box(color: colors.tertiary)[
  Never call `predict` before `fit`.
]
```

Call `fit(x, y)` first.

Never call `predict` before `fit`.

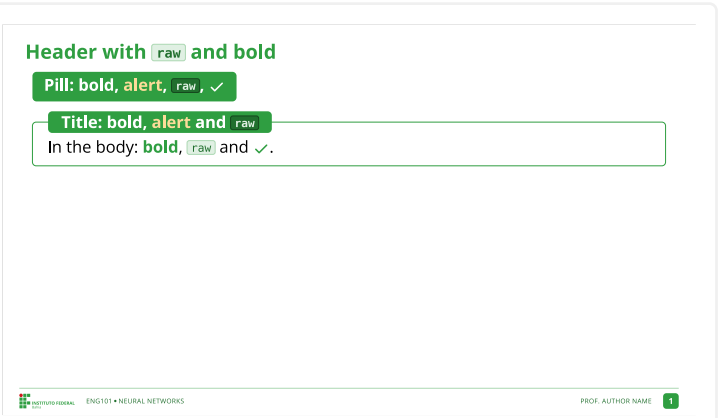
3.4 Colors by context

Bold text, alerts, highlights, underlines, arrows, chips and icons read the color of what contains them: the box color inside an `emph-box`, white or amber inside a pill, where the pill color would hide them.

```
== Header with `raw` and *bold*

=== Pill: *bold*, ~~alert~~, `raw`, #icon("check")

#emph-box(title: [Title: *bold*, ~~alert~~ and `raw`])[
  In the body: *bold*, `raw` and #icon("check").
]
```



3.5 Colored emphases

Bold text named after the color families of Section 7. A typical use is a label that opens a sentence.

`primary()`

```
primary(
  body content
) -> content
```

Bold text in IF green: the same as `#alert` (Touying's method) and as the `~~text~~` shorthand. Inside a pill (a `===` subtitle, an exercise header, the title of an `emph-box()`) it turns light amber, where green would vanish.

```
A #primary[definition], the same as ~~this shorthand~~.
```

A **definition**, the same as **this shorthand**.

Argument

body — `content`

Text to emphasize.

`secondary()`

```
secondary(
  body content
) -> content
```

Bold text in amber. It is weak on white: meant for short labels and dark backgrounds.

```
A #secondary[remark] in amber.
```

A **remark** in amber.

Argument

body — `content`

Text to emphasize.

`tertiary()`

```
tertiary(
  body content
) -> content
```

Bold text in red, for warnings and problems.

```
#tertiary[Caution:] the gradient vanishes.
```

Caution: the gradient vanishes.

Argument

body `content`

Text to emphasize.

`quaternary()`

```
quaternary(
  body content
) -> content
```

Bold text in blue, for examples and side remarks.

```
#quaternary[Note:] the series converges.
```

Note: the series converges.

Argument

body `content`

Text to emphasize.

3.6 Escapes

A backslash on the sign keeps a shorthand literal. An escaped `>` is only recognized when a word follows it.

```
\==not highlighted== and \~not underlined\~

\> not a box

-\> not a list

\"not a quotation\"
```

==not highlighted== and ~not underlined~

> not a box

-> not a list

"not a quotation"

4 Lists and terms

4.1 Arrow lists

Consecutive lines starting with `->` become one list with a green arrow. The list has to start the paragraph: a blank line comes before the first `->`. An arrow in the middle of a line and the math arrow are left alone.

```
-> consequence
-> another consequence

Here a -> b stays text, and $a -> b$ is math.
```

→ consequence
→ another consequence

Here a -> b stays text, and $a \to b$ is math.

`arrows()`

```
arrows(
  color: colors.primary  color
  body  content
) -> content
```

Arrow lists with the arrow in another color, for everything inside the body. The `-> item` shorthand itself needs no function: consecutive lines starting with `->` become one list with a green arrow (the box color inside an `emph-box()`). The list must start the paragraph, and `-\> item` escapes it.

```
#arrows(color: colors.quaternary)[
  -> blue here
  -> and here
]
```

→ blue here
→ and here

Argument

`color: colors.primary` `color`

Color of the arrows.

Argument

`body` `content`

Content with `-> item` paragraphs.

4.2 Bullet lists

Three levels of amber markers: a filled square, a rotated hollow square and a small circle.

```
- first level
  - second level
    - third level
- first level again
```

- first level
  - second level
    - third level
- first level again

4.3 Numbered lists

Numbers are always green and bold: `1.`, `a)`, `i.` by level. `#set enum(numbering: "a)")` starts at `a)` instead and keeps the colors. Typst only knows `+` and `1.` as enum markers in markup, so `a)` at the start of a line is text.

```
+ first
  + nested
    + deeper
+ second

#[
  #set enum(numbering: "a)")
  + starts at a)
    + then i.
]
```

1. first
   a) nested
      i. deeper
2. second

a) starts at a)
   i. then i.

4.4 Terms

```
/ Epoch: one pass over the training set.
/ Batch: the samples of one update.
```

Epoch: one pass over the training set.
Batch: the samples of one update.

5 Boxes and quotes

5.1 Emphasis boxes

`emph-box()`

```
emph-box(
  color: colors.primary  color
  title: none  none  content
  body  content
) -> content
```

Emphasis box. Without a title it has a thin border; with a title, a badge sits over the top border. Inside the box the `-` markers, the `->` arrow, `*bold*`, `==highlight==` and `#underline` take the box color; an explicit `#arrows(color:)`, `#highlight(fill:)` or `#underline(stroke:)` wins. Colors by convention: primary for definitions, quaternary for advantages, tertiary for problems, secondary for remarks.

```
#emph-box(color: colors.quaternary,
title: [Pros])[
  - Fast and *simple*
  - Works with ==highlights==
]
```

Pros

- Fast and **simple**
- Works with highlights

Shorthand, always in the primary color: a paragraph whose lines start with `>`; a first line `> == Title` gives the title. Only inline content fits the shorthand. For a framed equation see `eq-box()`.

Argument

`color: colors.primary` `color`

Color of the border and badge, and of the bold text, markers, arrows and highlights inside.

Argument

`title: none` `none` | `content`

Badge over the top border; `none` gives the thin-border box.

Argument

`body` `content`

Content of the box.

```
#emph-box[A definition, in the primary
color.]

#emph-box(color: colors.tertiary, title:
[Problem])[
  The gradient *vanishes* and ~stalls~.

  -> the arrow is red too
]
```

A definition, in the primary color.

Problem

The gradient **vanishes** and ~~stalls~~.

→ the arrow is red too

5.2 The box shorthand

A paragraph whose lines start with `>` is a box in the primary color; a first line `> == Title` gives the title.

```
> A box without title, with *bold* and $x^2$.

> == Definition
> A box with a title.
> The second line flows on.
```

A box without title, with **bold** and x^2.

Definition

A box with a title. The second line flows on.

Limits of the shorthand

Only inline content fits: lists and code blocks need `#emph-box`, and `>` before a block equation is an `eq-box`. A loose `>` at the start of a paragraph is read as a box: write `$a > b$`, or escape it (Section 3.6).

5.3 Equation boxes

eq-box( )

```
eq-box(
  color: auto  auto  color
  body  content
) -> content
```

Equation box: a light frame around a key equation (the `key-eq` helper of the lecture decks). It goes **inside** the equation, like the `boxeq` of the bookly package, so the equation keeps its centering, spacing and numbering, and a part of an equation can be framed too. The body is written as math, between dollar signs; it is set in display style whatever the spacing around it. A box around the whole equation is padded generously, a box around a part of one tightly, and in both cases the frame holds tall fractions and the limits of sums.

It also works in a line of text or in an inline equation: there the body is in text style and the box does not change the line spacing.

```
The output #eq-box[$y = phi(v)$] and the
mean #eq-box[$1/N sum_i y_i$].
```

The output $\boxed{y = \varphi(v)}$ and the mean $\boxed{\frac{1}{N}\sum_i y_i}$.

```
$ #eq-box[$v = bold(w)^top bold(x)$] $

$ y = #eq-box(color: colors.quaternary)
[$phi(v)$] + b $
```

$$\boxed{v = \boldsymbol{w}^\top \boldsymbol{x}}$$

$$y = \boxed{\varphi(v)} + b$$

Shorthand, always in the primary color: `>` in front of a block equation, on the same line or alone on the line before it.

```
> $ E = 1/(2N) sum_i (y_i - hat(y)_i)^2 $
```

$$E = \frac{1}{2N}\sum_i (y_i - \hat{y}_i)^2$$

Argument

`color: auto` — `auto` | `color`

Base color of the fill and border; `auto` is green, or the box color inside an `emph-box()`.

Argument

`body` — `content`

The equation, as math: `[$x = 2$]` or `($x = 2$)`.

Why the dollar signs inside

A content block inside an equation is markup, not math: `#eq-box[x = 2]` would be the upright text “x=2”, and an underscore or an asterisk in it would not even parse. `eq-box` refuses such a body with a hint. `> $x$`, with an inline equation, is an ordinary box: the shorthand takes a block equation.

5.4 Quotations

`quote-box()`

```
quote-box(
  color: colors.secondary  color
  body  content
) -> content
```

Quotation box: a vertical bar on the left. Typst’s `#quote(attribution: [Author])[…]` is shown as a `quote-box` too, with the attribution on the right.

```
#quote-box(color: colors.primary)[
  All models are wrong, but some are
  useful.
]
```

> All models are wrong, but some are useful.

Shorthand: a paragraph made only of `"quoted"` runs, one per line; each run becomes a box and the colors cycle secondary, primary, quaternary, tertiary. Quotes inside a sentence are left alone.

Argument

`color: colors.secondary` `color`

Color of the bar.

Argument

`body` `content`

The quotation.

```
"All models are wrong"
"but some are useful"

A sentence with a "quotation" inside.

#quote(attribution: [George Box])[
  All models are wrong.
]
```

> All models are wrong
>
> but some are useful

A sentence with a “quotation” inside.

> All models are wrong.
>
> — George Box

6 Columns

`cols()`

```
cols(
  columns: auto  auto  array
  gutter: 1em  length
  divider: false  bool
  ..bodies  content
) -> content
```

Columns: a thin wrapper over Touying's `cols` (a `grid`), with an optional divider. Touying's own `side-by-side` (no divider) is still available.

```
#cols(divider: true, columns: (1fr, 2fr))
[
  Left
][
  -> the shorthands work
  -> inside a column
]
```

Left

→ the shorthands work
→ inside a column

Argument

`columns: auto` — `auto` | `array`

Column widths, as Touying accepts them: `auto` (equal widths) or an array such as `(1fr, 2fr)`.

Argument

`gutter: 1em` — `length`

Space between the columns.

Argument

`divider: false` — `bool`

A green vertical bar (2.2pt) between the columns; the gutter is then used as inset.

Argument

`..bodies` — `content`

One content block per column.

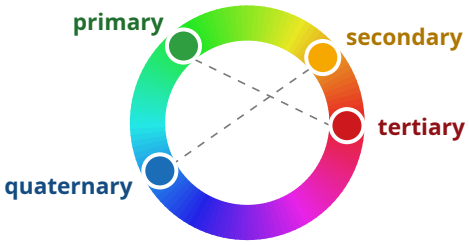
```
#cols[Left][Middle][Right]
```

Left Middle Right

Touying's `side-by-side` (no divider) is still available, and `#slide(composer: (1fr, 2fr))[..][..]` lays out the bodies of a whole slide.

7 Colors and logo

The palette starts from the brand of the Institutos Federais. Its two colors, green and red, are the classic complementary pair of the painter's color wheel; the theme adds the other one, amber and blue, which sit almost opposite to each other and fill the gaps the brand colors leave. The four bases are thus spread around the wheel, and each has a role: green carries the identity, amber marks and highlights, red is for exercises and blue for examples.



Family	Base	Hue	Origin
`primary`	`#2F9E41`	130°	IF green (Pantone 362 C)
`secondary`	`#F7A800`	41°	complement of the blue
`tertiary`	`#CD191E`	358°	IF red (Pantone 187 C)
`quaternary`	`#1C6DB8`	209°	complement of the amber

The green and the red are the HEX values printed in the brand manual of the Institutos Federais (2015).

Five families of seven tones, the four above and a neutral gray. The light tones serve as fills of highlights and boxes, the dark ones as text on white: IF green is a mid-tone, so small green text reads better in `primary-dark`.

	`-darkest`	`-darker`	`-dark`	base	`-light`	`-lighter`	`-lightest`
`primary`	#0C2810	#15471D	#216F2E	#2F9E41	#6DBB7A	#ACD8B3	#E0F0E3
`secondary`	#3E2A00	#6F4C00	#AD7600	#F7A800	#F9C24C	#FCDC99	#FEF2D9
`tertiary`	#330607	#5C0B0D	#901115	#CD191E	#DC5E62	#EBA3A5	#F8DDDD
`quaternary`	#071B2E	#0D3153	#144C81	#1C6DB8	#6099CD	#A4C5E3	#DDE9F4
`neutral`	#000000	#2B2B2B	#555555	#808080	#AAAAAA	#D5D5D5	#FFFFFF

The tones are derived from the base in RGB: `darken(75%)`, `darken(55%)`, `darken(30%)`, the base, `lighten(30%)`, `lighten(60%)` and `lighten(85%)`. The neutral family follows Touying's default instead, from black to white.

```
#highlight(fill: colors.tertiary-lighter)[red fill]
and #text(fill: colors.primary-dark)[dark green text].
```

red fill and dark green text.

7.1 The logo

The brand belongs to the Instituto Federal da Bahia and is not covered by the license of the package. The files in `assets/` are the official vector artwork, with the colors of the brand manual: `ifba-logo` is the complete horizontal version, the one the IFBA asks for outside the institution, and `ifba-logo-short` the short one. The theme itself draws the lettering in IF green (`logo-green`), in the footer and on the cover, where it sits better next to the green texts; that is a choice of this unofficial theme, not a version of the brand. On a dark background, use the white negative version (`logo-white`). In either case keep the proportions, leave one module (a square of the symbol) of clear space around the logo and do not use parts of the symbol on their own.

`logo-green()`

```
logo-green(
  logo  str
) -> str
```

A logo with its lettering in IF green instead of black, as the theme shows it in the footer: the green lettering sits better next to the green rules and texts of the slide. It is a choice of this theme, not an official version of the brand: `ifba-logo` and `ifba-logo-short` keep the official colors.

```
#image(bytes(logo-green(ifba-logo-
short)), height: 2em)
```



Argument

`logo` `str`

SVG source of the logo: `ifba-logo` or `ifba-logo-short`.

`logo-white()`

```
logo-white(
  logo  str
) -> str
```

Negative version of a logo: every fill becomes white, which is the monochrome negative version of the brand manual, for dark backgrounds.

```
#block(fill: colors.primary-dark,
inset: .5em, image(bytes(logo-white(ifba-
logo)), height: 2em))
```



Argument

`logo` `str`

SVG source of the logo: `ifba-logo` or `ifba-logo-short`.

`colors` `dictionary`

The theme palette: the two colors of the Institutos Federais brand (green and red, as printed in its 2015 brand manual), the complementary pair that completes the color wheel (amber and blue) and a neutral gray. The families are named after Touying's `config-colors` keys (`primary`, `secondary`, `tertiary`, `neutral`) plus `quaternary`. Each family has 7 tones, `<family>-darkest`, `-darker`, `-dark`, the base, `-light`, `-lighter` and `-lightest`, so `colors.primary-lighter`, `colors.quaternary-dark` etc. exist. The whole dictionary is passed to `config-colors`, hence the same keys are available as `self.colors.*`.

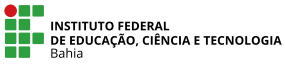
Family	Base	Role
`primary`	`#2F9E41`	IF green: titles, `->` markers, tables, `#alert`/`#primary`, pills
`secondary`	`#F7A800`	amber: `-` markers, `#secondary`, quotes; `-lighter` is the highlight
`tertiary`	`#CD191E`	IF red: `#tertiary`, exercises
`quaternary`	`#1C6DB8`	blue: `#quaternary`, examples
`neutral`	`#808080`	`-darkest` is the bold/italic text, `-lightest` the background

`ifba-logo` `str`

Official IFBA logo as an SVG string, in its complete horizontal version (“Instituto Federal de Educação, Ciência e Tecnologia / Bahia”), the one the IFBA asks for outside the institution. The title slide shows it in white (see `logo-white()`).

The brand belongs to the Instituto Federal da Bahia and is not covered by the license of the package.

```
#image(bytes(ifba-logo), height: 2em)
```



`ifba-logo-short` `str`

Short horizontal version of the IFBA logo (“Instituto Federal / Bahia”) as an SVG string, rendered in the footer with green lettering (see `logo-green()`): at that size the complete version would not be readable.

```
#image(bytes(ifba-logo-short), height:
2em)
```



8 Tables, figures, math, code and icons

8.1 Tables

The header row is green with white text, and formulas in it turn white too.

```
#table(
  columns: 3,
  table.header([Model], [Layers],
  [$eta$]),
  [MLP], [3], [0.1],
  [CNN], [8], [0.01],
)
```

Model	Layers	η
MLP	3	0.1
CNN	8	0.01

8.2 Figures and math

A figure caption shows its body only, without “Figure N”. Math uses New Computer Modern Math, and block equations sit closer to the text than Typst’s default. For a framed key equation, see `eq-box` in Section 5.

```
#figure(
  rect(width: 60%, height: 3em, fill:
  colors.primary-lightest),
  caption: [The caption, without a
  number.],
)

The update rule is
$ bold(w) <- bold(w) - eta nabla
E(bold(w)) $
```


The caption, without a number.

The update rule is

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla E(\boldsymbol{w})$$

8.3 Code

Inline `raw` is the chip of Section 3. Code blocks go through codly: a gray box, gray line numbers, no zebra, never split across pages. The theme re-exports codly, and its `local` wraps codly’s for per-block settings; `#codly-range(..)` and `#no-codly[..]` work as usual. The languages with a label are `python`, `typst`, `bash` and `text` (labelled “saída”).

````
== Code blocks

```python
def step(w, grad, lr=0.1):
    return w - lr * grad
```

#local(display-name: true,
highlights: ((line: 2, fill:
colors.secondary-lighter),))[
  ```python
  for epoch in range(10):
      w = step(w, grad(w))
  ```
]

#local(number-format: none)[
  ```bash
  typst compile slides.typ
  ```
]
````



`local()`

```
local(
  body content
  ..args any
) -> content
```

Per-block code settings: codly's `local`, re-exported by the theme so that its code box and `raw` handling also apply inside (this `local` shadows codly's). Typical uses: `#local(number-format: none)[…]` hides the line numbers, `#local(display-name: true)[…]` shows the language label and `#local(highlights: ((line: 2, fill: colors.secondary-lighter),))[…]` highlights a line. codly's `codly-range` and `no-codly` need no wrapper.

Argument

`body` `content`

Content with the code blocks.

Argument

`..args` `any`

Any argument of codly's `codly` function.

`code-languages` `dictionary`

Languages known to codly in code blocks. The `name` is what `#local(display-name: true)[…]` shows as a label; `text` is labelled "saída" (output), in Portuguese like the other user-facing defaults of the theme.

8.4 Icons

`icon()`

```
icon(
  name str
  color: auto auto color
  size: 1em length
  baseline: 25% relative
) -> content
```

An icon of the Octicons set (320 names), already wrapped in a `box` for inline use. An unknown name is a compile error.

```
#icon("check-circle", color:
colors.quaternary) Done,
#icon("alert") attention and a big
#icon("rocket", size: 1.6em, color:
colors.primary)
```

Done, attention and a big

Argument

`name` `str`

Name of the icon, e.g. `"check"`, `"alert"`, `"light-bulb"`, `"mark-github"`.

Argument

`color: auto` `auto` | `color`

Fill color. With `auto` it follows the context like bold text: white inside a pill (a `===` subtitle, an exercise header, the title of an `emph-box()`), the box or title color inside `emph-box()` and `==`, black elsewhere.

Argument

`size: 1em` `length`

Width and height of the icon.

Argument

`baseline: 25%` `relative`

Baseline of the box; the default aligns the icon with the text.

```
#emph-box(color: colors.tertiary)[
  #icon("alert") takes the box color.
]
```

⚠ takes the box color.

The 320 names, from Octicons (© GitHub Inc., MIT license):

accessibility	accessibility-inset	alert	alert-fill	apps
archive	arrow-both	arrow-down	arrow-down-left	arrow-down-right
arrow-left	arrow-right	arrow-switch	arrow-up	arrow-up-left
arrow-up-right	beaker	bell	bell-fill	bell-slash
blocked	bold	book	bookmark	bookmark-slash
briefcase	broadcast	browser	bug	cache
calendar	check	check-circle	check-circle-fill	checkbox
checklist	chevron-down	chevron-left	chevron-right	chevron-up
circle	circle-slash	clock	clock-fill	cloud
cloud-offline	code	code-of-conduct	code-review	code-square
codescan	codescan-checkmark	codespaces	columns	command-palette
comment	comment-discussion	container	copilot	copilot-error
copilot-warning	copy	cpu	credit-card	cross-reference
dash	database	dependabot	desktop-download	device-camera
device-camera-video	device-desktop	device-mobile	devices	diamond
diff	diff-added	diff-ignored	diff-modified	diff-removed
diff-renamed	discussion-closed	discussion-duplicate	discussion-outdated	dot
dot-fill	download	duplicate	ellipsis	eye
eye-closed	feed-discussion	feed-forked	feed-heart	feed-issue-closed
feed-issue-draft	feed-issue-open	feed-issue-reopen	feed-merged	feed-person
feed-plus	feed-public	feed-pull-request-closed	feed-pull-request-draft	feed-pull-request-open
feed-repo	feed-rocket	feed-star	feed-tag	feed-trophy
file	file-added	file-badge	file-binary	file-code
file-diff	file-directory	file-directory-fill	file-directory-open-fill	file-directory-symlink
file-moved	file-removed	file-submodule	file-symlink-file	file-zip
filter	fiscal-host	flame	fold	fold-down
fold-up	gear	gift	git-branch	git-commit
git-compare	git-merge	git-merge-queue	git-pull-request	git-pull-request-closed
git-pull-request-draft	globe	goal	grabber	graph
hash	heading	heart	heart-fill	history
home	horizontal-rule	hourglass	hubot	id-badge
image	inbox	infinity	info	issue-closed
issue-draft	issue-opened	issue-reopened	issue-tracked-by	issue-tracks
italic	iterations	kebab-horizontal	key	key-asterisk
law	light-bulb	link	link-external	list-ordered

list-unordered	location	lock	log	logo-gist
logo-github	mail	mark-github	markdown	megaphone
mention	meter	milestone	mirror	moon
mortar-board	move-to-bottom	move-to-end	move-to-start	move-to-top
multi-select	mute	no-entry	north-star	note
number	organization	package	package-dependencies	package-dependents
paintbrush	paper-airplane	paperclip	passkey-fill	paste
pencil	people	person	person-add	person-fill
pin	pin-slash	pivot-column	play	plug
plus	plus-circle	project	project-roadmap	project-symlink
project-template	pulse	question	quote	read
redo	rel-file-path	reply	repo	repo-clone
repo-deleted	repo-forked	repo-locked	repo-pull	repo-push
repo-template	report	rocket	rows	rss
ruby	screen-full	screen-normal	search	server
share	share-android	shield	shield-check	shield-lock
shield-slash	shield-x	sidebar-collapse	sidebar-expand	sign-in
sign-out	single-select	skip	skip-fill	sliders
smiley	sort-asc	sort-desc	sparkle-fill	sponsor-tiers
square	square-fill	squirrel	stack	star
star-fill	stop	stopwatch	strikethrough	sun
sync	tab-external	table	tag	tasklist
telescope	telescope-fill	terminal	three-bars	thumbsdown
thumbsup	tools	tracked-by-closed-completed	tracked-by-closed-not-planned	trash
triangle-down	triangle-left	triangle-right	triangle-up	trophy
typography	undo	unfold	unlink	unlock
unmute	unread	unverified	upload	verified
versions	video	webhook	workflow	x
x-circle	x-circle-fill	zap	zoom-in	zoom-out

9 Touying features

The theme sits on Touying and leaves its features alone. Animations work on every element of the theme: covered content stays faintly visible under a veil in the page color.

```
== Animations

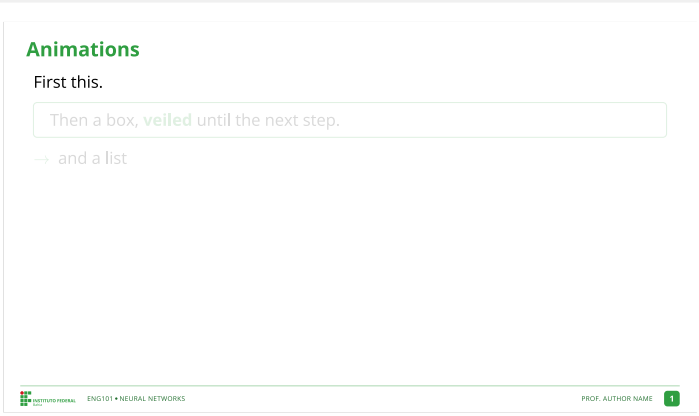
First this.

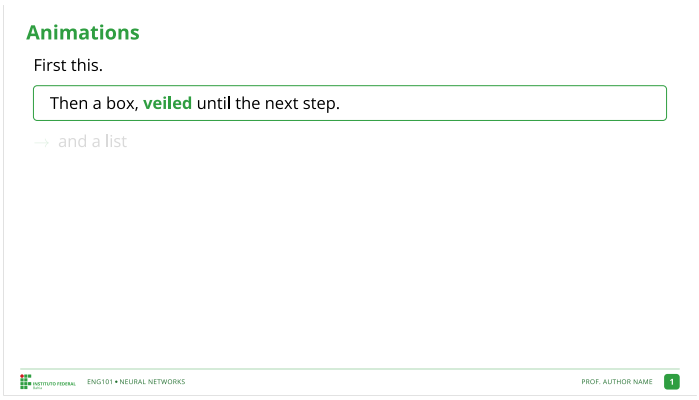
#pause

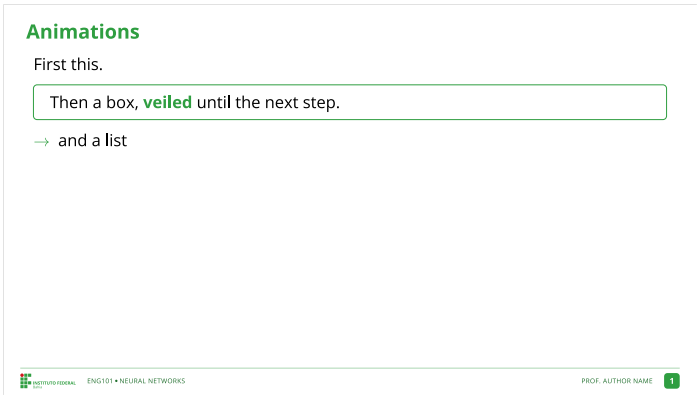
> Then a box, *veiled* until
the next step.

#pause

-> and a list
```







Keep `#pause` and `#uncover`/`#only` in separate paragraphs: within one paragraph Touying reorders the covered part on the first subslide.

Available without any wrapper from the theme:

Feature	Functions
Animations	`#pause`, `#meanwhile`, `#jump`, `#uncover`, `#only`, `#alternatives`, `#item-by-item`, `#effect`, `#touying-raw`
Notes and handouts	`#speaker-note` (it attaches to the previous slide), `#handout-only`, `config-common(handout: true)`

Feature	Functions
Layout	`#slide(composer: (1fr, 2fr))`, `#cols`, `components.adaptive-columns`, `components.full-width-block`, `utils.fit-to-width`, `utils.fit-to-height`
Overflow	`config-common(breakable: false, clip:, detect-overflow:)`
Reuse	`#touying-recall`, `touying-get-config`, `components.progressive-outline`

Slides of other Touying themes that this one does not have: `outline-slide`, `focus-slide`, `ending-slide` and `matrix-slide`.

10 Index of definitions

The reference of every function and variable sits in the chapter that introduces it; it is generated from the doc-comments of the package with tidy. Names that start with an underscore are internal and left out.

ifba.typ

`empty-slide()` p. 12
`example-slide()` p. 9
`exercise-slide()` p. 8
`ifba-theme()` p. 3
`new-section-slide()` p. 12
`slide()` p. 11
`title-slide()` p. 4

components.typ

`arrows()` p. 17
`cols()` p. 22
`emph-box()` p. 19
`eq-box()` p. 20
`icon()` p. 27
`local()` p. 27
`primary()` p. 15
`quaternary()` p. 16
`quote-box()` p. 21
`secondary()` p. 15
`tertiary()` p. 15

constants.typ

`code-languages` p. 27
`colors` p. 24
`ifba-logo` p. 24
`ifba-logo-short` p. 25
`logo-green()` p. 24
`logo-white()` p. 24